

Programming for Analytics

Week 2: Input & Output · Boolean Logic, Functions, Modules, Scope

Day 2 at The Daily Grind: the café learns to listen, prints an honest receipt, and remembers its revenue overnight.

Romain Claret · University College Dublin, 2026

Where you are

Week 2 of 10

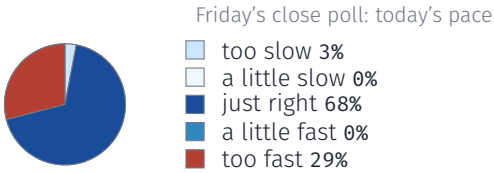
You can already: programs · `print()` · strings · math · variables · types · booleans · the float wall

Today you add: f-strings & formatting · `input()` · files (`open`/`read`/`write`) · `sys.argv` · comparisons & boolean logic · **functions** · scope · modules

Still ahead: decisions & loops (W3) · lists/dicts/sets/tuples (W4) · memory & unbreakable code (W5) · numpy + pandas + **your project** (W6, a double week) · classes (W7) · functional + GUIs (W8) · complexity (W9) · testing & exam prep (W10)

How last week landed

Self-check 1 closed last night. `70` of you took it.



Your close poll: asked above everything else: *why the float wall is off*, three slides on. Your other Week-1 questions: the **Q&A** page, under Feedback on Brightspace.

Self-check 1, the hardest question:

```
State the output of: print(0.1 + 0.1 + 0.1)
```

§0 · Outline: what's in this file

§	Section	Café / concept	Page
1	Talking to people (output)	the wall falls: f-strings, `.format`, the receipt table	8
2	Listening (`input()`)	the café takes a real order	14
3	Remembering (files)	revenue survives closing time	17
4	Starting up (program arguments)	`sys.argv`, and why they matter	23
	Break		24
5	Comparing (boolean logic)	the register answers questions (but can't act)	25
6	Functions	the receipt, refactored	31
7	Scope	why `revenue` isn't global	37
8	Modules	`register_sayings.py`: two files, one program	39
9	Exam & exercises · checklist		42

This week's café feature: a real (fixed-form) order, an aligned receipt, and revenue that survives overnight. **New concepts:** f-strings & `str.format()`; `input()`; file I/O; `sys.argv`; comparisons & `and`/`or`/`not`; functions; scope; modules.

Where our café is, and the debt we owe

Last week the café printed its first receipt, and ended on a bug I *promised* we'd fix:

```
3 lattes: 9.600000000000001
```

The value is fine; the **display** is naive. Today we clear that debt before the first section is out, and then the café learns three new abilities: **listening** (input), **remembering** (files), and **organizing itself** (functions).

You asked: why `0.1 + 0.1 + 0.1` is not `0.3`

1 · A tenth never ends in binary; a float keeps 53 bits and rounds the last one up.

`1/3` in base 10: `0.3333333333...` never ends → cut → carry a tiny error

`1/10` in base 2: a float is 64 bits: 1 for the sign, 11 to place the point, 52 digits, and the leading 1 is free

`0.000` `1` `1001 1001 1001 1001 ··· 1001 1001 1001 1001` `1001 1001 ...` rounded away

53 kept bits: the 1 of `0001`, then 13 groups of 4 — stored: `0.1000000000000000555...`

the dropped tail is worth 0.6 of the last kept bit, so it rounds up, as 0.666… → 0.67: `1001` becomes `1010`

2 · Add three, and the sum rounds up to the next tick.



3 · Python prints the shortest decimal that names a tick. `0.1` prints as `0.1`; the sum's tick has no shorter name than `0.30000000000000004`. Same story for `3.20 * 3`.

For the exam: **a tenth has no exact binary form, so floats carry tiny errors that can show when you add or multiply.** Display fix: three slides on. Rounding values: Week 4.

You asked, part two: why `0.1 + 0.1` prints `0.2`

1 · Doubling is free. `0.1 + 0.1` is `0.1` with the point moved one place: the same 53 bits, nothing dropped, so no new error, and it lands exactly on the tick `0.2` names.

`0.1`	$1\ 1001\ 1001\ \ldots\ 1001\ 1010 \times 2^{-4}$	
`0.1 + 0.1`	$1\ 1001\ 1001\ \ldots\ 1001\ 1010 \times 2^{-3}$	same 53 bits, point moved: exactly `0.2`'s tick

2 · Two sizes meet. `0.2 + 0.1` adds numbers of different sizes; lined up, the exact sum needs one more bit than a float holds. It falls exactly halfway between two ticks, and the tie rule picks the upper one here.

3 · So the error is not always there. It shows only when a rounding lands off the tick you expect: `0.1 * 2` → `0.2`, `0.1 * 4` → `0.4`, but `0.1 * 3` → `0.30000000000000004`, and `10/3` three times → `10.0`. You cannot see it coming, so never depend on the last digits.

```
>>> 0.1 + 0.1
0.2
>>> 0.1 + 0.1 + 0.1
0.30000000000000004
```

§1 · CONCEPT Programs eat data and produce data

A program is a machine for **data**: some comes in, some goes out, like a mathematical function $x \to f \to y$.

In	Out
keyboard	screen
files	files
program arguments	printers, actions…

Last week our café had **no input at all** (everything hard-coded) and one output (the screen). This week we fill the whole table.

§1 · CAFÉ Three ways to print data, and the fix

```
>>> latte_price = 3.20
>>> lattes = 3
>>> print("Total:", latte_price * lattes)          # commas: mixed types, auto-spaces
>>> print("Total: " + str(latte_price * lattes))   # convert + concatenate (Week 1)
>>> print(f"Total: {latte_price * lattes}")        # f-strings: expressions in braces
```

All three print `9.600000000000001`. Now, the fix:

```
>>> print(f"Total: {latte_price * lattes:.2f}")
Total: 9.60
```

`:.2f` = *"format as a float, two digits after the point."* **The wall is dead.** The value in memory is still imprecise; we **display money like a register**.

§1 · CONCEPT f-strings

```
>>> students = 70
>>> lecturers = 1
>>> place = "room"
>>> print(f"There are {students + lecturers} people in this {place} now.")
There are 71 people in this room now.
```

- Prefix the string with `f`, put **any expression** inside `{...}`.
- The expression is evaluated **when the string is created:** remember that for the drill three slides on.
- This is the style I'll use most. But you must also read the older one →

§1 · CONCEPT `str.format()`: the older sibling

```
>>> print("There are {} people in this {} now.".format(71, "room"))
There are 71 people in this room now.
>>> print("Order: {0} {1} {2}".format(1, 2.0, "3"))     # by position
Order: 1 2.0 3
>>> print("Chaos: {2} {0} {1}".format(1, 2.0, "3"))     # any order (count from 0!)
Chaos: 3 1 2.0
>>> print("{:d} = {:f} = {:e}".format(3, 3, 3))         # force a type
3 = 3.000000 = 3.000000e+00
```

Type codes: `d` integer · `f` float (decimal) · `e` float (exponential) · `s` string. You'll meet `.format()` in older code and in exam questions: read both dialects.

Format specs earn their keep: the receipt table

A spec can set **width**, **precision**, **alignment** and **fill**, which is exactly a receipt:

```
>>> print(f"{' The Daily Grind ':=^34}")
>>> print(f"{'latte':<12} x {3:>2} {9.6:>8.2f}")
```

```
======== The Daily Grind =========
latte        x  3     9.60
```

- `:<12` left-aligns in 12 characters · `:>2` right-aligns in 2
- `:>8.2f` right-aligns, width 8, two decimals · `:=^34` centers, filling with `=`

One line of spec replaced all of Week 1's manual spacing. *(Heads-up: the free books cover f-string basics but **not** these specs; the slides are your reference here.)*

§1 · TRY IT Predict, then run (3 min)

```
>>> x = 23; phrase = f"I am {x} years old"; x = 24
>>> print(phrase); print(f"I am {x} years old")   # ← what prints? (creation-time!)

>>> print("|{0:-^12}|".format("Center"))
>>> print("|{0:{1}<12}|".format("Left", "-"))      # the fill char as an argument!
>>> print("|{0:{1}>12}|".format("Right", "-"))
>>> print("{s} = {n}".format(n=3, s="three"))      # named placeholders
```

The first line is a classic: `phrase` was built **when x was 23**, so rebinding `x` later doesn't reach back (Week 1's rebinding rule, wearing a new hat).

§2 · CONCEPT Listening: `input()`

```
name = input("Your name: ")
print(f"Hello, {name}!")
```

- Prints the prompt, waits for the keyboard, **returns what was typed as a `str`**, always a string, without exception.
- Compact form: `print(f"Hello, {input('Your name: ')}!")` is legal, but keep it readable.
- **Save the result in a variable** or it's gone (garbage collection, Week 1).

The `input()` trap: you'll hit this tonight

```
year = input("Which year did you first visit us? ")
print(2026 - year, "years of loyalty!")
```

```
TypeError: unsupported operand type(s) for -: 'int' and 'str'
```

`input()` returned the **text** `"2019"`, not the number. Read the error message carefully: it names both types. The fix is Week 1's conversion:

```
print(2026 - int(year), "years of loyalty!")
```

Rule: convert immediately: `qty = int(input("How many? "))`.

§2 · CAFÉ The café takes its first real order

```
cashier = input("Cashier name: ")
print(register_sayings.welcome(cashier))

espresso_qty = int(input("How many espressos? "))
latte_qty = int(input("How many lattes? "))
muffin_qty = int(input("How many muffins? "))
```

(That `register_sayings.welcome(...)` line is §8's story: two files, one program. Until then, read it as "print the greeting".)

One question per product. Clumsy, and we know it: with twenty products this is twenty hard-coded lines, and there's no way to say *"keep asking until the customer is done"*. **That's a loop, and loops are next week.** Feel the itch; note the shape.

§3 · CONCEPT Remembering: data lives in files

Everything on your disk is a file: documents, spreadsheets, programs, and our revenue.

- File I/O works like keyboard/screen I/O. In fact, C gave us three famous "files": **stdin** (the keyboard), **stdout** (the screen), **stderr** (the error channel). You *will* hear "print to stdout" in the wild.
- The **extension** hints at the content: `.txt` text · `.csv` comma-separated text · `.py` Python source · `.xlsx` Excel · `.jpg` image.
- Underneath, every file is **binary:** text files are character codes (Week 1's ASCII story), encoded (UTF-8, Unicode…).

§3 · CONCEPT Opening a file

```
menu = open("menu.txt", "r")     # or just open("menu.txt"): "r" is the default
```

Mode	Meaning	after writing twice
`"r"`	**read** (default)	n/a
`"w"`	**write:** starts the file fresh (overwrites!)	`line 2` (the last only)
`"a"`	**append:** adds to the end	`line 1` `line 2`
`"r+"`	read *and* write	not demoed

The variable holds a **file object**, not the contents! It's a *pointer* to the file plus a **bookmark** of where you're reading. (And relax: garbage-collecting the object never deletes the file itself.)

(`menu.txt` is on Brightspace with this week's files: replay this section at home.)

§3 · CONCEPT Reading: the bookmark moves

```
>>> menu = open("menu.txt")
>>> print(menu.readline())     # line 1
>>> print(menu.readline())     # line 2: the object REMEMBERS where you are
>>> print(menu.read(21))       # exactly 21 characters
>>> whole = menu.read()        # ...everything from here to the end (careful: can be huge)
>>> menu.close()
>>> menu.readline()
ValueError: I/O operation on closed file.
```

- `readline()` → one line per call, in order.
- `read(n)` → n characters; `read()` → the whole rest.
- **Close what you open.** After `close()`, the file object refuses to work.



the bookmark, after each call

The bookmark, explicitly: `tell()` and `seek()`

```
>>> book = open("menu.txt")
>>> print(book.tell())        # 0: the bookmark, in bytes (ASCII: bytes = chars)
>>> line = book.readline()
>>> print(book.tell())        # 27: it moved
>>> book.seek(4)              # jump the bookmark to character 4
>>> print(book.readline())    # reads from there
```

`tell()` reads the bookmark, `seek()` moves it. Rarely needed at the café; but examinable, and it explains *why* two `readline()` calls give two different lines.

§3 · CAFÉ Revenue that survives closing time

```
revenue_file = open("revenue.txt", "w")        # "w": start the file fresh
revenue_file.write(f"{total:.2f}")             # write() wants a string -- f-strings deliver
revenue_file.close()

log = open("receipts.txt", "a")                # "a": append to the running log
log.write(f"{total:.2f}\n")
log.close()
```

Two modes, two jobs: **"w"** for *the* revenue figure (fresh each close), **"a"** for the receipt log (history accumulates). And the trap that catches everyone:

```
>>> values = open("scratch.txt", "w")
>>> values.write(3.14)
TypeError: write() argument must be str, not float
```

`write()` takes **strings only:** convert first (`str()` or an f-string).

Proof it survived: write, quit, read back

```
revenue_file = open("revenue.txt", "r")       # read it back: proof it survived
saved = revenue_file.readline()
revenue_file.close()
print(f"Revenue on file: {saved}")
```

Live demo, the full circle: run the café → close it → **kill the terminal** → run a one-line reader → `17.00` is still there. That's the difference between a **variable** (dies with the program) and a **file** (survives it).

Safe reading of a file that might NOT exist (first morning, no revenue yet) needs Week 5's exceptions. Until then we only read files we just wrote.

§4 · CONCEPT Starting up: program arguments

```python
# w02_argv_start.py (on Brightspace: the road not taken, kept runnable)
import sys
cashier = sys.argv[1]                  # argv[0] is the program's own name
starting_cash = float(sys.argv[2])     # arguments arrive as strings -> convert
print(f"Register open. {cashier} starts the day with {starting_cash:.2f} in the drawer.")
```

```
$ python3 w02_argv_start.py Ada 40.0
Register open. Ada starts the day with 40.00 in the drawer.
```



- `sys.argv` = the **argument vector**, **always as strings** (convert!).
- Two import styles: `from sys import argv` (then bare `argv`) or `import sys` (then `sys.argv`, *the more pythonic form*; you always see where a name came from).
- That `['...', '...']` thing? It's called a **list:** Week 4 makes lists ours.

Break

10–15 minutes

*When we come back: the register learns to **answer questions** (booleans), the receipt gets **refactored into functions**, and the café becomes a **two-file program**.*

Think about it: the register now listens and remembers. What yes-or-no questions should it be able to answer about an order?

☎ *Answer on PollEv when we come back.*

§5 · CONCEPT Comparing data

```
>>> revenue = 17.0
>>> target = 150.0
>>> hit_target = revenue > target     # the answer is a VALUE...
>>> print(hit_target, type(hit_target))
False <class 'bool'>
```

A comparison **produces a `bool` object:** Week 1's `True`/`False`, now earned rather than typed. Save it in a variable like any other value (or it's garbage-collected).

Math	$<$	$\leq$	$>$	$\geq$	$=$	$\neq$
Python	`<`	`<=`	`>`	`>=`	`==`	`!=`

Strings compare too: `"espresso" == "latte"` → `False`. (Order comparisons like `"iPhone" > "iPad"` are legal but rarely what you mean: stick to `==`/`!=` for text.)

The classic: == compares, = assigns

```
>>> product1 = "espresso"
>>> product2 = "latte"
>>> comparison = product1 == product2
>>> print(comparison)
False                                   # a question, answered

>>> comparison = product1 = product2
>>> print(comparison)
latte                                   # ...an assignment CHAIN. No question was asked.
```

The second line quietly made `comparison` and `product1` point at `"latte"`, and `"espresso"` was garbage-collected. One `=` short, no error message, wrong program.

§5 · CONCEPT Combining questions: `and`, `or`, `not`

*Did we serve 10 or more customers today, **and** was the revenue above 150?*

```
>>> customers = 12
>>> revenue = 145.0
>>> print(customers >= 10 and revenue > 150.0)
False
```

Pretty much English:

- `and`: **all** parts must be true;
- `or`: **at least one** must be true;
- `not`: flips it, `not (interruptions > 5)`.

The precedence trap: always use parentheses




```
>>> uni = "UCD"
>>> age = 18
>>> print(uni == "UCD" or uni == "DCU" or uni == "TCD" and age > 21)
True        # ...an 18-year-old passed an over-21 check?!
```

Priority is **not > and > or**, so the line really means `(uni=="UCD") or (uni=="DCU") or (uni=="TCD" and age>21)`: the age check only guards TCD.

```
>>> print((uni == "UCD" or uni == "DCU" or uni == "TCD") and age > 21)
False       # what we meant
```

When in doubt, and even when not, use parentheses. They cost nothing.

Truth tables

x	y	x and y	x or y
False	False	False	False
False	True	False	True
True	False	False	True
True	True	**True**	True

x	not x
False	True
True	False

`and` = *all* true · `or` = *at least one* true · `not` = invert. These tables are exam material, and in Week 9 they come back as the branches we count.

The register answers questions… and hits a wall




```
big_order = total > 10.0
busy_start = espresso_qty + latte_qty + muffin_qty >= 5
print(f"Big order (over 10.00)?  {big_order}")
print(f"Busy start (5+ items)?   {busy_start}")
```

```
Big order (over 10.00)?  True
Busy start (5+ items)?   True
```

The register can now **compute a decision**… and can't **act** on it. *"IF it's a big order, apply a discount"*: we can say the first half in Python and not the second.

That sentence's missing half is `if`, and it's next week. (This is exactly last week's float-wall move: feel the limit, then we remove it.)

§6 · CONCEPT Functions: same logic, different data

You've been *calling* functions since minute one: `print()`, `type()`, `len()`, `input()`, `open()`, `int()`… Now we **make our own**:

```
def line_total(price, quantity):
    return price * quantity
```

- `def` + name + `(parameters)` + `:`, then the **indented body**.
- `return` hands a value back to the caller.
- Why bother? The same question, asked many times, with different data? Write the logic **once**, call it forever.

§6 · CONCEPT Indentation is the syntax

```
def big_order_check(total):
    threshold = 10.0            # ← 4 spaces: "I belong to this def"
    return total > threshold
```

- An indented line says *“I’m a sub-level of what came before.”*
- Python doesn’t fix the amount; **the standard is 4 spaces**, and it’s what this course uses. Mixing tabs and spaces = misery. Set VS Code to spaces and forget it.
- In the interactive shell, the body shows the `...` prompt; finish with a blank ENTER.

§6 · CONCEPT Zero or more in, zero or more out

```
def hello():                            # no inputs, no return (it just DOES something)
    print("Hello!")

def cube(nbr):                          # one in, one out
    return nbr * nbr * nbr

def euros_and_cents(amount):            # one in, TWO out
    whole = amount // 1.0
    return whole, amount - whole
```

```
>>> euros, cents = euros_and_cents(17.50)   # euros = 17.0, cents = 0.5
```

Multiple returns are just values separated by commas, and they are examinable. *(Here `cents` is the fraction of a euro; for `17.50` it is `0.5`, so × `100` is `50` cents. The café's receipt does that multiply.)*

§6 · CAFÉ The receipt, refactored

Week 1's receipt was copy-paste. Now it's **one rule, reused**:

```
def receipt_line(product, price, quantity):
    """One aligned receipt row -- the format specs do the layout work."""
    return f"{product:<12} x {quantity:>2} {line_total(price, quantity):>8.2f}"
```

```
print(f"{' The Daily Grind ':=^34}")
print(receipt_line("espresso", espresso_price, espresso_qty))
print(receipt_line("latte", latte_price, latte_qty))
print(receipt_line("muffin", muffin_price, muffin_qty))
print("=" * 34)
```

A new product = **one line**, not four. Full file: `cafe_w2.py`. *(`receipt_line` calls `line_total`: functions build on functions.)*

§6 · TRACE Follow a call by hand

```python
def line_total(price, quantity):
    return price * quantity

def receipt_line(product, price, quantity):
    return f"{product:<12} x {quantity:>2} {line_total(price, quantity):>8.2f}"

print(receipt_line("latte", 3.20, 3))
```

Step	Where	What happens
1	`print(...)`	needs `receipt_line("latte", 3.20, 3)` first
2	`receipt_line`	`product="latte"`, `price=3.20`, `quantity=3`
3	`line_total`	`price=3.20`, `quantity=3` → returns `9.6...`
4	back in the f-string	`9.6...` formatted `:>8.2f` → `"    9.60"`
5	back in `print`	the full row appears

"State the output of this program" on the exam is exactly this table. Practice the walk.

§6 · TRY IT Your turn (4 min)

```
>>> def greet(name): print(f"Hello, dear {name}!")
...
>>> def leave(): print("I'm leaving now, goodbye!")
...
>>> greet(input("What's your name? ")); leave()

>>> def area_of_ellipse(semi_major, semi_minor):
...     pi = 3.14
...     return pi * semi_major * semi_minor
...
>>> print(area_of_ellipse(1.0, 1.0))
>>> print(pi)                    # ← predict THIS one before you run it
```

That last line is the cliff-edge into our next topic…

§7 · CONCEPT Scope: where a name lives

```python
def hello():
    msg = "Hello, world!"      # msg is LOCAL: born in the function, dies with it
    print(msg)

hello()
print(msg)                     # NameError: name 'msg' is not defined
```

- A variable born **inside** a function exists only there (that's why `print(pi)` blew up).
- A **global** (born at file level, *before* the call) is readable inside functions; born *after* the call runs? `NameError` (Week 5 turns this into a file-layout rule); but a local of the same name **shadows** it, and assigning inside a function makes a *new local*, it does **not** change the global.
- The `global` keyword *can* force it… **avoid globals unless strictly necessary.**

Design & why: `revenue` is not global

The lazy design: make `revenue` global, let every function add to it. It works, until you can't tell **who changed the money**, or test any piece alone.

The café's rule (and your project's, in week 6):

> *Functions **receive** what they need and **return** what they produce.* `total = line_total(price, qty)`*: data flows through the front door.*

This is the week-2 seed of the biggest design idea in this course: *one clear owner for every piece of state.* Remember this slide in Week 7.

§8 · CONCEPT Modules: don't reinvent the wheel

Python ships with **batteries included:** collections of functionality called modules:

- `sys`: system interaction (our `sys.argv`) · `os`: operating system
- `math`: proper math · `random`: random numbers · `datetime`: dates

```
>>> import math
>>> print(math.sqrt(2))          # module.member: you can SEE where names come from
1.4142135623730951
```

Two styles: `import sys` → `sys.argv` (pythonic, traceable) or `from sys import argv` (shorter, but where did `argv` come from?). Thousands more modules exist to download; that's Python at its most generous. And you can write your own →

§8 · CAFÉ Our first two-file program

```python
# register_sayings.py -- the cafe's first MODULE
def welcome(name):
    return f"Welcome to The Daily Grind, {name}!"

goodbye = "Thank you, come again!"
```

```python
# cafe_w2.py
import register_sayings
print(register_sayings.welcome(cashier))    # module.function()
print(register_sayings.goodbye)             # module.variable
```

- A module is **just a Python file:** import it by filename, no `.py`.
- **A module is executed when imported:** so it should contain *declarations* (functions, variables), not surprise `print()` calls. The café is officially multi-file.

Where the café is now

Runnable snapshot, **`cafe_w2.py`** (+ `register_sayings.py`): takes a real order, prints an aligned receipt (**`9.60`**, rest in peace, wall), answers boolean questions, and saves revenue that survives closing time.

The walls we hit **on purpose** this week:

- the order form is **hard-coded:** one question per product, no "until done" → **loops, W3**;
- `big_order` is `True` and we **can't act on it** → **`if`, W3**;
- reading a file that might not exist would crash → **exceptions, W5**.

Next week the register gets a **menu loop** and its first real **decisions**, and we draw the program *before* we write it.

Exam pointers

The exam can ask you to: state the output of format specs (`{x:>8.2f}`) and function calls (the trace table!); spot the `input()`-returns-a-string TypeError, the `=` vs `==` bug, and the `and`/`or` precedence trap; explain local vs global scope; complete a one-line function; pick the right file mode (`w` versus `a`); unpack a function's two return values; import your own module.

Exercises & self-study

(optional; full map on Brightspace):

- **Self-check 2** → Brightspace tonight: about fifteen anonymous minutes, closes Thursday 23:59. We open next week on whatever the class lost.
- **Practical sheet** → Brightspace (Monday's practical works through it; bring the laptop with Python + VS Code installed).
- **Project groups:** the group form is on Brightspace; form your group of five, all from your practical session, before the **end of week 5**.
- **py4e** *(free, recommended)*: Ch 2 (input) + Ch 4 (functions) + Ch 6 (strings) + the opening of Ch 7 (files): videos + auto-graded exercises.
- **ATBS** *(free alternative)*: Ch 4 (functions, the best free *scope* coverage) + Ch 8 (strings) + Ch 10 (files) + Ch 12 (command line / `argv`). · **LPTHW** *(owners)*: Ex 5–17, 18–28 (23–24 & 26 optional).
- *Heads-up:* py4e Ch 7's exercises and both books' projects use **loops** (next week); and py4e Ch 4's last exercises need `if`. Read now, park those exercises until W3. ATBS uses `with open(...)`: fine, but our course style is plain `open()`/`close()`.

This week's checklist: tick before the practical

- ☐ I can format money correctly: f-strings and `.format()` with width/precision specs.
- ☐ I can take `input()`, and I convert it immediately, because it is always a string.
- ☐ I can `open`/`readline`/`read`/`write`/`close` a file, and pick the right mode (`r`/`w`/`a`).
- ☐ I can read `sys.argv` and say what `argv[0]` is.
- ☐ I can build boolean expressions with comparisons + `and`/`or`/`not`, with parentheses.
- ☐ I can write a function with parameters and (multiple) return values.
- ☐ I can explain local vs global scope, and why we avoid globals.
- ☐ I can split a program across two files and import my own module.

Anything unticked → that section of this deck, then the practical sheet.

Next week: flow control: the menu loop, decisions, and drawing programs before writing them.